

# ARDOISE — Carnet de crédit numérique pour les boutiquiers de quartier

---

## Document de ressources pour mémoire de fin d'études

**Projet** : Ardoise (package Android `com.ardoise.credit`) **Dépôt** : <https://github.com/User26003/ardoise-project> **Stack** : Flutter 3.35.4 · Dart 3.9.2 · Hive 2.2.3 · Provider 6.1.5 · speech\_to\_text 7.x **Volume** : ~4 900 lignes de Dart, 16 fichiers source, 9 tests unitaires, 2 commits (v1.0)

---

### Comment utiliser ce document

Il est organisé pour suivre la structure classique d'un mémoire d'informatique / génie logiciel (Licence ou Master) : contexte et problématique → état de l'art → analyse des besoins → conception → réalisation → tests → discussion → perspectives. Chaque partie contient (a) le contenu factuel du projet réel, (b) des éléments de justification scientifique, (c) des pistes de rédaction et de réflexion critique. Les encadrés « ► Pour le mémoire » donnent des conseils de rédaction ou des questions à approfondir. Les extraits de code sont tirés du dépôt réel.

---

## Table des matières

1. Propositions de titre et de problématique
  2. Contexte et justification
  3. État de l'art
  4. Analyse des besoins
  5. Conception
  6. Réalisation technique
  7. Algorithmes clés
  8. Tests et validation
  9. Résultats et discussion
  10. Limites et perspectives
  11. Plan de rédaction proposé
  12. Bibliographie et webographie
  13. Annexes
-

# 1. Propositions de titre et de problématique

## 1.1 Titres possibles

- *Conception et réalisation d'une application mobile hors ligne de gestion du crédit informel pour les commerces de proximité au Bénin : le cas d'Ardoise*
- *Numérisation du « cahier de l'ardoise » : une approche mobile-first, vocale et hors connexion pour les boutiquiers de quartier*
- *Inclusion numérique des micro-commerces : conception d'un carnet de crédit mobile à saisie vocale adapté aux contraintes de connectivité et d'alphabétisation en Afrique de l'Ouest*

## 1.2 Problématique centrale

**Comment concevoir un outil numérique de gestion du crédit informel qui soit réellement adopté par des boutiquiers de quartier, compte tenu de leurs contraintes spécifiques : téléphones bas de gamme, connectivité intermittente, hétérogénéité des niveaux d'alphabétisation, et pratiques commerciales orales et relationnelles ?**

## 1.3 Questions de recherche secondaires

1. Quels sont les mécanismes actuels de gestion du crédit (« l'ardoise ») dans les boutiques de quartier béninoises, et quelles pertes économiques et sociales génèrent-ils ?
2. Dans quelle mesure une **saisie vocale en français** (avec un parseur tolérant aux formulations locales) réduit-elle la friction d'utilisation par rapport à une saisie textuelle ?
3. Une architecture **offline-first** (stockage local Hive, aucune dépendance serveur) est-elle suffisante pour le cas d'usage, et à quel prix (perte de données, absence de synchronisation) ?
4. Comment modéliser la notion de **retard de paiement** pour qu'elle reflète la pratique réelle (date promise oralement par le client) plutôt qu'un seuil arbitraire ?
5. Quels canaux de rappel (SMS via Intent, appel, Mobile Money) sont les plus pertinents et acceptables socialement ?

## 1.4 Hypothèses

- **H1** : la saisie vocale au format « *Nom, article, montant* » est apprise en moins de 2 minutes par un utilisateur sans formation.
- **H2** : la visibilité immédiate du « total dehors » modifie le comportement d'octroi de crédit du boutiquier.
- **H3** : un rappel SMS rédigé poliment et mentionnant la date convenue augmente le taux de recouvrement sans dégrader la relation client.
- **H4** : un stockage 100 % local est perçu comme plus digne de confiance qu'un stockage cloud par les utilisateurs cibles.

► **Pour le mémoire** : formuler 3 à 5 hypothèses maximum et prévoir pour chacune un moyen de vérification (enquête terrain, test utilisateur chronométré, comparaison avant/après sur 2-4 semaines).

## 2. Contexte et justification

### 2.1 Le crédit informel dans le commerce de proximité

Dans les quartiers de Cotonou, Porto-Novo, Parakou ou Abomey-Calavi, les « boutiques du coin » et les « bonnes dames » (vendeuses de produits alimentaires) sont le premier point d'approvisionnement des ménages. Une part significative des ventes se fait **à crédit**, par confiance, avec un règlement différé (fin de semaine, jour de paie, fin du mois). Ce crédit est noté « à l'ardoise » : historiquement sur une ardoise à craie, aujourd'hui dans un cahier d'écadier.

**Dysfonctionnements observés** (à confirmer par votre enquête terrain) :

| Problème                               | Conséquence                                                                  |
|----------------------------------------|------------------------------------------------------------------------------|
| Cahier perdu, mouillé, déchiré         | Perte totale de l'information, pertes financières irrécouvrables             |
| Écriture illisible, oubli de noter     | Contestations, sous-évaluation des créances                                  |
| Pas de total par client ni global      | Le boutiquier ne sait pas « combien il a dehors » ; sur-exposition au risque |
| Pas de trace de la date                | Impossibilité de savoir depuis quand un client doit ; retards non détectés   |
| Rappel uniquement oral, en face à face | Gêne sociale, conflits, clients qui évitent la boutique                      |
| Aucune preuve en cas de litige         | Rapport de force défavorable au boutiquier                                   |

### 2.2 Enjeux

- **Économique** : le crédit non recouvré est une perte directe de trésorerie pour des commerces à très faible marge.
- **Social** : l'ardoise est un mécanisme de solidarité de quartier ; l'outil doit le préserver et non le rigidifier.
- **Inclusion numérique** : la cible utilise des smartphones Android d'entrée de gamme (1-2 Go de RAM, Android 8-12), souvent en données mobiles limitées.
- **Innovation locale** : les solutions existantes sont conçues pour le Nigeria ou le Kenya (anglophones, écosystème M-Pesa) et ne sont pas adaptées au contexte francophone béninois (FCFA, MTN MoMo / Moov Money / Celtiis, prénoms fon/yoruba/dendi, formulations en français local).

### 2.3 Terrain d'étude suggéré

- 10 à 20 boutiquiers dans 2 quartiers contrastés (ex. un quartier populaire dense et un quartier périurbain).
- Entretiens semi-directifs (30 min) + observation du cahier existant.
- Grille : nombre de clients à crédit, encours moyen estimé, fréquence des pertes, canal de rappel, possession et usage d'un smartphone, langue de travail.

► **Pour le mémoire** : les chiffres « 45 000 F dehors chez 12 personnes » utilisés dans l'app sont illustratifs. Remplacez-les par des données de votre enquête pour crédibiliser la justification.

## 3. État de l'art

### 3.1 Solutions existantes de « digital ledger » en Afrique et en Asie

| Solution              | Pays    | Modèle                                                 | Points forts                             | Limites pour le Bénin                       |
|-----------------------|---------|--------------------------------------------------------|------------------------------------------|---------------------------------------------|
| <b>OkCredit</b>       | Inde    | Ledger client/fournisseur, rappels SMS/WhatsApp, cloud | Très grande adoption (>10 M), simplicité | Hindi/anglais, dépend du cloud, roupies     |
| <b>Khatabook</b>      | Inde    | Ledger + paiements UPI intégrés                        | Écosystème complet                       | Idem, non adapté                            |
| <b>Kippa</b>          | Nigeria | Bookkeeping + paiements + prêts                        | Fintech complète                         | Anglais, naira, connexion requise           |
| <b>Bumpa / Sabi</b>   | Nigeria | Gestion commerce + e-commerce                          | Fonctions riches                         | Trop complexe pour une boutique de quartier |
| <b>Pezesha / Tala</b> | Kenya   | Scoring crédit via M-Pesa                              | Intégration Mobile Money native          | Centré prêt, pas ledger informel            |
| <b>Weebi</b>          | Sénégal | Caisse + stock pour boutiques                          | Francophone, FCFA                        | Payant, orienté caisse plutôt que crédit    |
| <b>Cahier papier</b>  | Partout | —                                                      | Gratuit, zéro apprentissage              | Toutes les limites de §2.1                  |

**Constat** : aucune solution ne combine (1) le français local, (2) le FCFA, (3) le hors ligne total, (4) la saisie vocale, (5) la notion de *date promise*.

### 3.2 Saisie vocale et interfaces pour publics faiblement lettrés

- Les travaux en **HCI4D** (Human-Computer Interaction for Development) montrent que la voix et les icônes réduisent la barrière d'usage pour les utilisateurs peu ou non lettrés (Medhi et al., 2011 ; Sherwani et al., 2009).

- Deux familles techniques : **reconnaissance embarquée** (Vosk/Kaldi, Whisper.cpp, moteur Google hors ligne) versus **API cloud** (Google Speech, Azure, Whisper API). Le compromis retenu dans Ardoise : le moteur système Android via `speech_to_text`, qui fonctionne **hors ligne si le pack de langue française est installé**, sans coût ni serveur.
- Défi propre au terrain : l'accent, le code-switching français/fon, les nombres dits « à la française » (*mille deux cents*) → nécessité d'un **parseur tolérant** (voir §7.1).

### 3.3 Architectures offline-first

- Principe : l'application est pleinement fonctionnelle sans réseau ; la synchronisation est optionnelle et asynchrone (Kleppmann et al., 2019, « Local-first software »).
- Choix de stockage sur Flutter : SQLite (`sqflite`, non disponible sur le Web), **Hive** (NoSQL clé-valeur, pur Dart, multiplateforme), Isar, ObjectBox. Hive retenu pour sa légèreté (<1 Mo), sa vitesse et sa compatibilité Web (utile pour les démonstrations).

### 3.4 Mobile Money en Afrique de l'Ouest

- Bénin : MTN Mobile Money, Moov Money, Celtiis Cash ; interopérabilité via la plateforme GIMAC/BCEAO en cours.
- Approche d'Ardoise : **pas d'intégration API** (complexe, payante, KYC) mais **enregistrement manuel** du mode de paiement (Espèces / Mobile Money) et insertion du numéro MoMo du boutiquier dans le SMS de rappel pour faciliter le règlement à distance.

► **Pour le mémoire** : l'état de l'art doit se conclure par un **tableau comparatif** et un paragraphe « positionnement » qui explicite la contribution originale d'Ardoise.

## 4. Analyse des besoins

### 4.1 Acteurs

- **Boutiquier / bonne dame** (utilisateur principal) : enregistre crédits et paiements, consulte, relance.
- **Client** (acteur externe) : reçoit des SMS, paie en espèces ou par MoMo. N'utilise pas l'app.
- **Système Android** (acteur technique) : moteur de reconnaissance vocale, application SMS, application Téléphone.

### 4.2 Besoins fonctionnels (implémentés en v1.0)

| ID    | Besoin                                                                                | Priorité | Statut    |
|-------|---------------------------------------------------------------------------------------|----------|-----------|
| BF-01 | Enregistrer un crédit (client, article, montant) par la voix                          | Haute    | [OK]      |
| BF-02 | Enregistrer un crédit par saisie manuelle                                             | Haute    | [OK]      |
| BF-03 | Créer un client à la volée (nom, téléphone)                                           | Haute    | [OK]      |
| BF-04 | Enregistrer un paiement (espèces / Mobile Money)                                      | Haute    | [OK]      |
| BF-05 | Voir le solde de chaque client et le total global « dehors »                          | Haute    | [OK]      |
| BF-06 | Consulter l'historique complet d'un client                                            | Haute    | [OK]      |
| BF-07 | Enregistrer une <b>date promise de remboursement</b>                                  | Haute    | [OK]      |
| BF-08 | Détecter automatiquement les clients en retard                                        | Haute    | [OK]      |
| BF-09 | Envoyer un rappel SMS pré-rédigé, poli, avec la date convenue                         | Haute    | [OK]      |
| BF-10 | Appeler le client en un geste                                                         | Moyenne  | [OK]      |
| BF-11 | Statistiques : jour, mois, 7 jours, top débiteurs, articles, répartition espèces/MoMo | Moyenne  | [OK]      |
| BF-12 | Filtrer : Tous / Aujourd'hui / En retard / Payé ; recherche par nom                   | Moyenne  | [OK]      |
| BF-13 | Modifier / supprimer un client ou une opération                                       | Moyenne  | [OK]      |
| BF-14 | Configurer le profil boutique et le seuil de retard                                   | Moyenne  | [OK]      |
| BF-15 | Onboarding au premier lancement                                                       | Basse    | [OK]      |
| BF-16 | Sauvegarde / restauration (fichier, cloud)                                            | Haute    | [v2] v2   |
| BF-17 | Notifications planifiées de rappel                                                    | Moyenne  | [v2] v2   |
| BF-18 | Export PDF / partage WhatsApp du relevé                                               | Moyenne  | [v2] v2   |
| BF-19 | Reconnaissance vocale en fon / yoruba                                                 | Basse    | Recherche |

## 4.3 Besoins non fonctionnels

| ID     | Exigence                                              | Solution                                                                                                          |
|--------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| BNF-01 | <b>Hors ligne total</b>                               | Hive local ; aucun appel réseau dans le code métier                                                               |
| BNF-02 | <b>Téléphones bas de gamme</b>                        | Release build, pas d'animations lourdes, pas de dépendances natives lourdes                                       |
| BNF-03 | <b>Simplicité</b> (3 gestes max pour noter un crédit) | Bouton micro central, feuille unique de saisie, montants rapides                                                  |
| BNF-04 | <b>Lisibilité</b>                                     | Gros chiffres, couleurs sémantiques (orange = doit, vert = payé, rouge = retard)                                  |
| BNF-05 | <b>Responsive</b> (320 px → tablette / web)           | Module <code>responsive.dart</code> , <code>Wrap</code> , <code>FittedBox</code> , <code>MaxWidth</code>          |
| BNF-06 | <b>Confidentialité</b>                                | Données sur l'appareil uniquement ; aucune collecte                                                               |
| BNF-07 | <b>Langue</b>                                         | Interface 100 % français, formats FCFA, dates <code>fr_FR</code>                                                  |
| BNF-08 | <b>Maintenabilité</b>                                 | Séparation modèles / services / provider / écrans ;<br><code>flutter analyze</code> sans erreur ; tests unitaires |

## 4.4 Cas d'utilisation principaux

UC-01 Noter un crédit par la voix

Acteur : Boutiquier

Pré-condition : app ouverte, micro autorisé

Scénario nominal :

1. Appuie sur le bouton micro central
2. Dit « Codjo, riz, 500 »
3. Le système transcrit, parse, pré-remplit client / article / montant
4. Si le client existe → il est associé ; sinon → proposé en création
5. (Optionnel) Renseigne « Il paie quand ? »
6. Valide « Noter sur l'ardoise »

Post-condition : transaction créée, solde mis à jour, liste rafraîchie

Alternatives :

- 3a. Rien compris → message d'erreur, saisie manuelle proposée
- 3b. Montant absent → bouton de validation désactivé jusqu'à saisie

UC-02 Enregistrer un paiement

UC-03 Envoyer un rappel SMS

UC-04 Fixer / modifier la date promise

UC-05 Consulter les statistiques

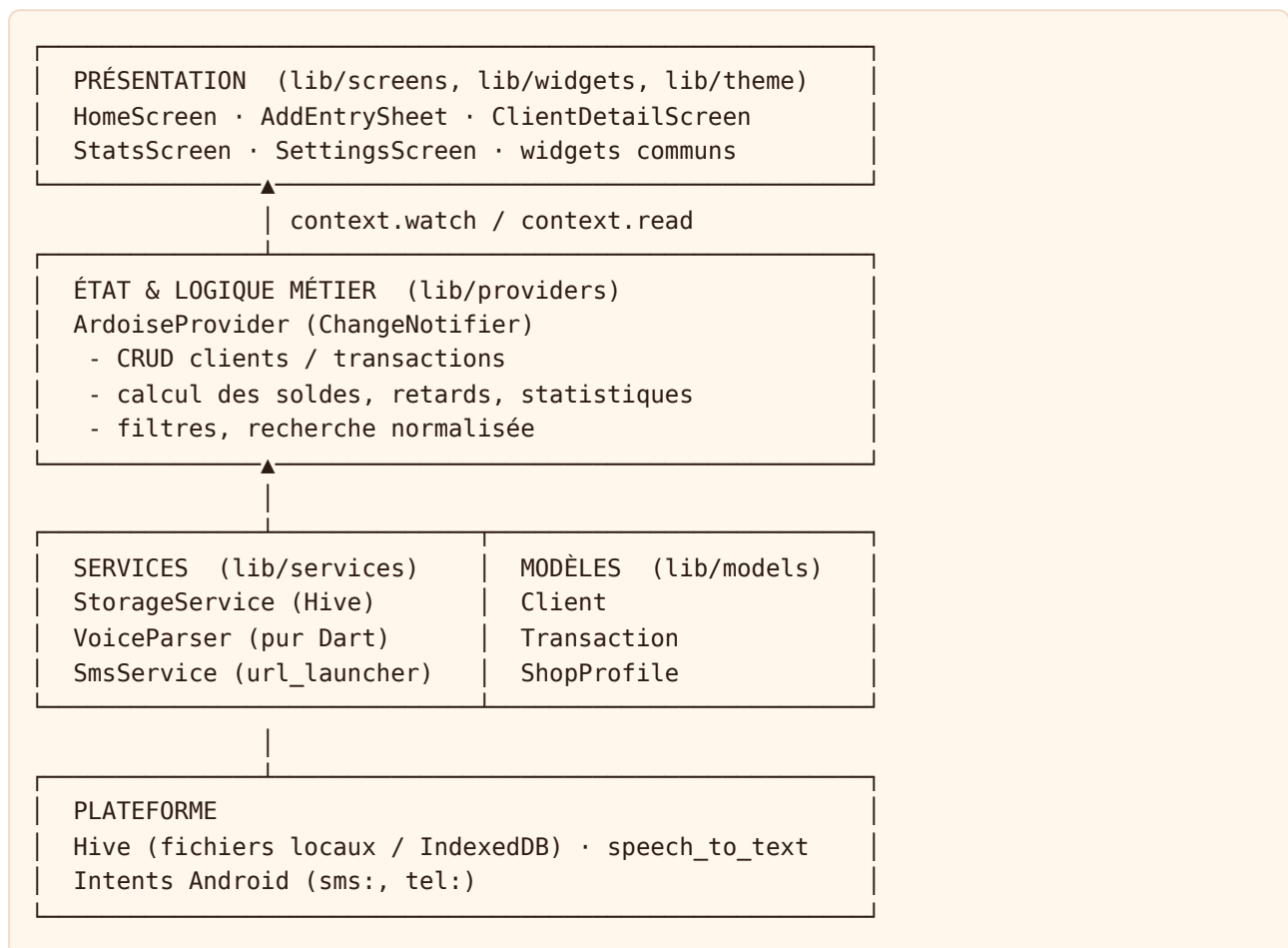
► **Pour le mémoire** : produire un **diagramme de cas d'utilisation UML** et 3 à 5 **diagrammes de séquence** (UC-01 est le plus démonstratif car il fait intervenir le

moteur vocal, le parseur, le provider et le stockage).

## 5. Conception

### 5.1 Architecture logicielle

Architecture **en couches** inspirée de MVVM, avec Provider comme couche de présentation d'état :



**Justifications :**

- **Provider** plutôt que Bloc/Riverpod : courbe d'apprentissage minimale, suffisant pour un état unique global, recommandé officiellement par Flutter.
- **Hive sans génération de code** : les entités sont sérialisées en `Map` ( `toMap` / `fromMap` ), ce qui évite `build_runner` et rend les migrations de schéma triviales (champs optionnels avec valeur par défaut — cf. ajout de `promiseDate` en v1.1 sans migration).
- **Parseur vocal en Dart pur** : testable unitairement, indépendant du moteur de reconnaissance (remplaçable par Vosk ou Whisper sans toucher au parseur).

### 5.2 Modèle de données



| Client                  | Transaction                                    | ShopProfile           |
|-------------------------|------------------------------------------------|-----------------------|
| id : String (UUID v4)   | id : String (UUID v4)                          | ownerName : String    |
| name : String           | clientId : String → Client                     | shopName : String     |
| phone : String          | type : credit   payment                        | phone : String (MoMo) |
| note : String           | amount : int (FCFA)                            | reminderDays : int    |
| createdAt : DateTime    | label : String (article)                       |                       |
| colorIndex : int        | method : cash   mobileMoney   other (nullable) |                       |
| promiseDate : DateTime? | date : DateTime                                |                       |
|                         | viaVoice : bool                                |                       |

**Choix de conception notables** : - Le **solde n'est pas stocké** : il est toujours *dérivé* ( $\Sigma$  crédits -  $\Sigma$  paiements). Cela garantit la cohérence et permet la suppression d'une opération sans recalcul manuel (principe de *source unique de vérité*). - Les montants sont des **entiers** (le FCFA n'a pas de centimes) → pas d'erreurs d'arrondi flottant. - `promiseDate` est portée par le **client**, pas par la transaction : dans la pratique, le client promet de régler *son ardoise*, pas une ligne précise.

Boîtes Hive : `clients`, `transactions`, `settings` (Clé `profile`, `onboardingDone`, `demoSeeded`).

### 5.3 Conception de l'interface (UX)

**Principes retenus** (issus du choix de design « Style 2 + profil du style 1 + dates ») :

1. **Un geste principal, toujours visible** : le bouton micro central (FAB docké), tap = écoute immédiate, appui long = saisie manuelle.
2. **Le chiffre qui compte en premier** : « TOTAL DEHORS » en très gros dans l'en-tête, avec le nombre de débiteurs et de retards.
3. **Codage couleur sémantique constant** : orange (crédit / doit), vert (payé / soldé), rouge (retard), ambre (promesse à venir).
4. **Identité visuelle locale** : dégradé orange-rouge et motif géométrique inspiré du **wax** (`WaxPatternPainter`, dessiné en `CustomPaint` pour ne charger aucune image).
5. **Date à côté du nom** : chaque ligne client montre la dernière activité et la promesse ; l'information temporelle n'est jamais cachée.
6. **Tolérance à l'erreur** : confirmation avant toute suppression ; possibilité de corriger la transcription vocale avant d'enregistrer.
7. **Accessibilité** : cibles tactiles  $\geq 44$  px, texte système limité entre  $0,85\times$  et  $1,3\times$  pour ne jamais casser la mise en page.

**Navigation** : 2 onglets seulement (Clients, Stats) + réglages accessibles depuis l'avatar. Choix délibéré de **réduire** la navigation pour un public peu habitué aux applis.

### 5.4 Règle métier : le retard de paiement

Formalisation (implémentée dans `ArdoiseProvider.isLate` et `ClientSummary.lateDays`) :

Soit un client  $c$  avec solde  $s(c)$ , date promise  $p(c)$  (optionnelle), dernière activité  $a(c)$ , et seuil global  $R$  (jours, réglage).

```
enRetard(c) =
  s(c) > 0 ∧ (
    p(c) ≠ ∅ ⇒ aujourd'hui > p(c)           -- retard dès le lendemain
    p(c) = ∅ ⇒ (aujourd'hui - a(c)) ≥ R      -- règle par défaut
  )

joursDeRetard(c) =
  si p(c) ≠ ∅ : max(0, aujourd'hui - p(c))
  sinon      : max(0, (aujourd'hui - a(c)) - R)
```

**Intérêt** : la date promise, information *négociée oralement* entre le client et le boutiquier, devient une donnée de première classe, ce qui rend le système fidèle à la pratique sociale au lieu de lui imposer un délai arbitraire.

## 6. Réalisation technique

### 6.1 Organisation du code

```
lib/
├─ main.dart                Bootstrap, thème, navigation racine, FAB micro
├─ models/models.dart       Client, Transaction, ShopProfile (+ sérialisation
Map)
├─ providers/ardoise_provider.dart État global, règles métier, statistiques, seed
démo
├─ services/
│   └─ storage_service.dart  Persistance Hive (3 boîtes)
│   └─ voice_parser.dart     Analyse des phrases dictées (pur Dart)
│   └─ sms_service.dart      Rappel SMS / appel via Intents
├─ screens/
│   └─ home_screen.dart      En-tête profil + total, filtres, liste clients
│   └─ add_entry_sheet.dart   Feuille micro + saisie manuelle + date promise
│   └─ client_detail_screen.dart Fiche client, promesse, actions, historique
│   └─ stats_screen.dart     Tableaux de bord
│   └─ settings_screen.dart  Profil, seuil de retard, onboarding
├─ widgets/common.dart      WaxHeader, ClientAvatar, StatTile, EmptyState...
├─ theme/app_theme.dart     Palette, ThemeData Material 3
├─ utils/
│   └─ formatters.dart       FCFA, dates relatives, libellés de promesse
│   └─ responsive.dart       Points de rupture, échelle, MaxWidth
test/widget_test.dart       9 tests unitaires
android/                    Package com.ardoise.credit, permissions, intents
web/                        Manifest PWA, thème
assets/icons/app_icon.png   Icône générée
```

### 6.2 Dépendances et justification

| Package                      | Version       | Rôle                          | Pourquoi                                                                         |
|------------------------------|---------------|-------------------------------|----------------------------------------------------------------------------------|
| provider                     | 6.1.5         | État                          | Simple, officiel, suffisant                                                      |
| hive + hive_flutter          | 2.2.3 / 1.1.0 | Stockage local                | Rapide, pur Dart, Web + Android, sans SQL                                        |
| speech_to_text               | ^7.0          | Reconnaissance vocale         | Utilise le moteur système (hors ligne possible), gratuit                         |
| url_launcher                 | 6.3.1         | Intents sms: / tel:           | Pas de permission SEND_SMS (respect vie privée), l'utilisateur garde le contrôle |
| intl + flutter_localizations | 0.20.2        | Formats fr_FR                 | Dates, nombres                                                                   |
| uuid                         | 4.5.1         | Identifiants                  | Uniques hors ligne, sans serveur                                                 |
| shared_preferences           | 2.5.3         | Réservé (préférences légères) | —                                                                                |

### 6.3 Persistance (extrait)

```
// storage_service.dart
Future<void> init() async {
  await Hive.initFlutter();
  _clients = await Hive.openBox('clients');
  _transactions = await Hive.openBox('transactions');
  _settings = await Hive.openBox('settings');
}

List<Client> getClients() =>
  _clients.values.map((e) => Client.fromMap(e as Map)).toList();

Future<void> saveClient(Client c) => _clients.put(c.id, c.toMap());
```

Désérialisation **défensive** (compatibilité ascendante des schémas) :

```
factory Client.fromMap(Map m) => Client(
  id: m['id'] as String,
  name: (m['name'] as String?) ?? '',
  promiseDate: m['promiseDate'] == null
    ? null
    : DateTime.fromMillisecondsSinceEpoch(m['promiseDate'] as int),
  ...
);
```

### 6.4 Reconnaissance vocale (extrait)

```

await _speech.listen(
    onResult: _onSpeechResult,
    listenOptions: SpeechListenOptions(
        localeId: 'fr_FR',
        listenFor: const Duration(seconds: 12),
        pauseFor: const Duration(seconds: 3),
        partialResults: true,
        listenMode: ListenMode.dictation,
    ),
);

```

Le résultat final est passé à `VoiceParser.parse()` puis **pré-remplit** le formulaire ; l'utilisateur **confirme** (principe « humain dans la boucle », indispensable vu le taux d'erreur de la reconnaissance sur des prénoms locaux).

## 6.5 Rappel SMS par Intent (extrait)

```

final uri = Uri(scheme: 'sms', path: phone, queryParameters: {'body': body});
await launchUrl(uri, mode: LaunchMode.externalApplication);

```

Le message s'adapte à la promesse : - « *Bonjour Codjo, petit rappel de Boutique Chez Mama Afi : votre ardoise est de 1 800 F. Comme convenu, merci de passer régler le 12 mars 2026. Mobile Money : 97 00 00 00* » - « *... La date convenue (10 mars 2026) est passée, merci de passer régler. ...* »

**Justification éthique** : l'app n'envoie jamais de SMS à l'insu de l'utilisateur ; elle ouvre l'application SMS pré-remplie. Pas de permission `SEND_SMS`, pas de coût caché, ton respectueux.

## 6.6 Responsive

```

class R {
    static bool isCompact(BuildContext c) => width(c) < 360; // petits téléphones
    static bool isWide(BuildContext c) => width(c) >= 700; // tablette / web
    static bool isShort(BuildContext c) => height(c) < 600; // paysage / clavier
    static double sp(BuildContext c, double v) => v * scale(c); // 0.9 → 1.1
}

```

Techniques : `Wrap` pour nom + badges de date, `FittedBox` pour les grands montants, `SliverGrid` 2 colonnes sur écran large, `MaxWidth` (640-900 px) pour le confort de lecture, `textScaler.clamp(0.85, 1.3)`.

## 6.7 Configuration Android

- `applicationId` / `namespace` : `com.ardoise.credit` ; `android:label="Ardoise"`.
- Permissions : `RECORD_AUDIO` (micro), `INTERNET` (nécessaire au moteur vocal si pack hors ligne absent), Bluetooth (casques).
- `<queries>` pour `RecognitionService`, `SENDTO sms:`, `DIAL tel:` (visibilité des packages Android 11+).

- Icône adaptative générée dans toutes les densités ( `mipmap-*` ).

## 7. Algorithmes clés

### 7.1 Parseur de phrases dictées ( `VoiceParser` )

**Entrée** : chaîne transcrite. **Sortie** : { `clientName?`, `label?`, `amount?`, `isPayment?` }.

1. Normaliser : minuscules, apostrophes → espaces
2. Détecter le PAIEMENT : tokeniser ; si un token sans accent ∈ {paye, payer, rembourse, remboursement, verse, donne, regle, solde} → `isPayment = true`, retirer le token
3. Extraire le MONTANT numérique : regex `(\d{1,3}(?:[.]\d{3})+|\d+)`  
→ prendre la DERNIÈRE occurrence (le montant est dit en fin de phrase), supprimer les séparateurs de milliers
4. Segmenter sur , ; / → si ≥ 2 segments : format « nom, article, montant »  
sinon : format phrase libre
5. Si pas de montant numérique : convertir les MOTS-NOMBRES français (un...seize, vingt, trente..., cent(s), mille, million) avec la grammaire additive/multiplicative : cent → ×100 ; mille → ×1000 puis accumulation  
ex. « mille deux cents » → 1000 + 2×100 = 1200
6. Filtrer les MOTS VIDES (a, à, pris, de, du, francs, fcfa, chez, doit...)
7. Client = premier segment / premier mot significatif (capitalisé)  
Article = deuxième segment / mots restants (ignoré si paiement)
8. Retourner le résultat ; l'UI cherche ensuite le client par nom NORMALISÉ (sans accents, préfixe ou inclusion) pour tolérer « Sènan » ≈ « senan »

**Complexité** : linéaire en nombre de mots. **Robustesse** : testée sur 5 formulations (voir §8). **Limites** : nombres composés complexes (« quatre-vingt-dix-sept mille ») partiellement gérés ; homonymes de clients non désambiguïsés (→ l'UI propose la sélection).

### 7.2 Calcul des résumés clients ( `summaryOf` )

Un seul passage sur les transactions du client : accumule `credited`, `paid`, `lastCredit`, `lastPayment`, `lastActivity`. Solde = `credited` - `paid`. Coût O(T) par client, O(C·T) pour la liste ; acceptable pour quelques centaines de clients et quelques milliers d'opérations (cas réel d'une boutique). Une optimisation par index `clientId` → `List<Transaction>` est prévue si besoin.

### 7.3 Détection du retard et promesses proches

Voir formalisation §5.4. `upcomingPromises` retourne les clients dont `daysToPromise` ∈ {0, 1} pour la bannière « X a promis de payer aujourd'hui / demain ».

### 7.4 Statistiques

- Agrégats temporels par bornes `[from, to]` : `creditedBetween`, `paidBetween`.

- Série 7 jours : liste de tuples (jour, crédits, paiements) rendue en histogramme CustomPaint-free (widgets FractionallySizedBox).
- Top débiteurs : tri décroissant sur solde, take(5).
- Top articles : agrégation par libellé normalisé (majuscule initiale), tri, take(5).

## 8. Tests et validation

### 8.1 Tests unitaires (9, tous passants)

| Groupe      | Test                                 | Vérifie                          |
|-------------|--------------------------------------|----------------------------------|
| VoiceParser | Codjo, riz, 500                      | format à virgules                |
| VoiceParser | Afi a pris de l'huile à 800 francs   | phrase naturelle, mots vides     |
| VoiceParser | Rachid a payé 2000                   | détection paiement (accent)      |
| VoiceParser | Nadège savon mille deux cents        | nombres en lettres               |
| VoiceParser | Kossi, gaz, 6 500                    | séparateur de milliers           |
| Retard      | promesse hier                        | daysToPromise = -1, lateDays = 1 |
| Retard      | promesse aujourd'hui                 | pas encore en retard             |
| Retard      | promesse future + 30 j sans paiement | la promesse prime                |
| Retard      | sans promesse                        | règle des R jours                |

Commande : flutter test · Qualité statique : flutter analyze → **0 problème**.

### 8.2 Tests manuels réalisés

- Parcours complet sur Web (Chrome) : onboarding → saisie vocale → création client → paiement → promesse → retard → rappel SMS (ouverture de l'app externe non testable sur Web).
- Responsive : 320 px, 360 px, 412 px, 768 px, 1280 px ; paysage ; texte système 200 %.

### 8.3 Protocole de validation terrain recommandé (à réaliser pour le mémoire)

1. **Test d'utilisabilité** (n = 8-12 boutiquiers) : 5 tâches (noter un crédit par la voix, enregistrer un paiement, trouver combien doit un client, fixer une date, envoyer un rappel). Mesurer : taux de réussite, temps, erreurs, échelle SUS (System Usability Scale).
2. **Étude longitudinale** (2-4 semaines, 5 boutiques) : comparer le cahier et l'app en parallèle ; mesurer le nombre d'opérations saisies, les écarts, le nombre de rappels envoyés, le recouvrement.
3. **Précision vocale** : 50 phrases par 5 locuteurs → taux de reconnaissance brute (WER) et taux de parsing correct (client + montant exacts).

4. **Entretiens post-usage** : perception de la confiance (données locales), acceptabilité sociale du SMS, souhaits.

► **Pour le mémoire** : c'est cette partie qui transforme un projet technique en travail de recherche. Même un petit échantillon bien documenté vaut mieux qu'aucune donnée.

## 9. Résultats et discussion

### 9.1 Résultats techniques obtenus

- Application fonctionnelle multi-plateforme (Android + Web de démonstration), **100 % hors ligne**.
- Saisie vocale en français avec parseur tolérant aux formulations locales (5 formats validés).
- Modélisation du retard fidèle à la pratique (date promise).
- Interface responsive de 320 px à l'écran d'ordinateur.
- Base de code analysée sans erreur, testée, versionnée sur GitHub.

### 9.2 Points de discussion

- **Voix vs texte** : la voix est rapide mais dépend du moteur système et du bruit du marché ; le formulaire pré-rempli + confirmation est le compromis retenu. Discuter la pertinence d'un moteur embarqué (Vosk) pour garantir l'hors-ligne sur tous les appareils, au prix de ~50 Mo de modèle.
- **Local-first vs cloud** : gain de confiance et de simplicité, mais **risque de perte de données** en cas de vol / casse du téléphone → la sauvegarde (BF-16) devient critique.
- **Rappel SMS manuel vs automatique** : l'ouverture de l'app SMS préserve le contrôle et la relation ; un envoi automatique serait plus efficace mais potentiellement mal perçu.
- **Transposabilité** : le code est paramétrable (devise, langue, opérateurs MoMo) pour le Togo, le Niger, le Burkina, la Côte d'Ivoire.

## 10. Limites et perspectives

### 10.1 Limites actuelles

1. Pas de sauvegarde / restauration → perte possible des données.
2. Reconnaissance vocale dépendante de Google et du pack français hors ligne ; pas de fon / yoruba / dendi.
3. Pas de notification planifiée (l'utilisateur doit ouvrir l'app pour voir les retards).
4. Pas d'intégration API Mobile Money (paiement enregistré manuellement).

5. Un seul utilisateur par appareil ; pas de multi-boutique ni de rôles.
6. Validation terrain non encore réalisée.

## 10.2 Feuille de route (perspectives de recherche et développement)

| Horizon | Évolution                                                                                     | Intérêt scientifique                            |
|---------|-----------------------------------------------------------------------------------------------|-------------------------------------------------|
| Court   | Sauvegarde chiffrée locale + export/import fichier, partage WhatsApp du relevé (PDF)          | Résilience des données en contexte local-first  |
| Court   | Notifications locales ( flutter_local_notifications ) le jour de la promesse                  | Effet des rappels sur le recouvrement           |
| Moyen   | Reconnaissance vocale embarquée (Vosk / Whisper tiny) avec vocabulaire de prénoms locaux      | Adaptation ASR aux langues et accents peu dotés |
| Moyen   | Synchronisation optionnelle chiffrée (Firebase / Supabase) avec résolution de conflits (CRDT) | Local-first multi-appareils                     |
| Moyen   | Intégration MTN MoMo API / Moov Money (paiement par lien, notification de réception)          | Interopérabilité fintech                        |
| Long    | Scoring de fiabilité client (historique de promesses tenues)                                  | Micro-crédit informel et données                |
| Long    | Interface en fon / yoruba avec pictogrammes et TTS                                            | HCI4D, inclusion                                |

## 11. Plan de rédaction proposé



Dédicaces · Remerciements · Résumé (FR/EN) · Sommaire · Listes des figures, tableaux, sigles

## INTRODUCTION GÉNÉRALE

Contexte, problématique, objectifs, hypothèses, méthodologie, plan

## PARTIE I – CADRE THÉORIQUE ET CONTEXTUEL

- Chap. 1 Le commerce de proximité et le crédit informel au Bénin (§2)
- Chap. 2 État de l'art : ledgers numériques, ASR, offline-first, MoMo (§3)
- Chap. 3 Méthodologie : enquête terrain, démarche de conception (§2.3, §8.3)

## PARTIE II – ANALYSE ET CONCEPTION

- Chap. 4 Analyse des besoins (acteurs, cas d'utilisation, exigences) (§4)
- Chap. 5 Conception (architecture, modèle de données, UX, règles métier) (§5)

## PARTIE III – RÉALISATION ET ÉVALUATION

- Chap. 6 Environnement et implémentation (§6)
- Chap. 7 Algorithmes : parseur vocal, retard, statistiques (§7)
- Chap. 8 Tests, validation et résultats (§8, §9)
- Chap. 9 Discussion, limites et perspectives (§10)

## CONCLUSION GÉNÉRALE

BIBLIOGRAPHIE (§12)

ANNEXES : captures d'écran, code source clé, guide d'installation, questionnaire d'enquête, grille SUS (§13)

**Volume indicatif** : 60–90 pages (Licence) ; 90–130 pages (Master).

## 12. Bibliographie et webographie

Vérifiez et complétez chaque référence selon la norme demandée par votre établissement (APA, IEEE, ISO 690).

### Ouvrages et articles

- Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). *Local-first software: You own your data, in spite of the cloud*. Onward! 2019, ACM.
- Medhi, I., Patnaik, S., Brunskill, E., Gautama, S. N., Thies, W., & Toyama, K. (2011). *Designing mobile interfaces for novice and low-literacy users*. ACM TOCHI, 18(1).
- Sherwani, J., Ali, N., Rosé, C. P., & Rosenfeld, R. (2009). *Orality-grounded HCID: Understanding the oral user*. Information Technologies & International Development, 5(4).
- Donner, J. (2015). *After Access: Inclusion, Development, and a More Mobile Internet*. MIT Press.
- Suri, T., & Jack, W. (2016). *The long-run poverty and gender impacts of mobile money*. Science, 354(6317).

- Collins, D., Morduch, J., Rutherford, S., & Ruthven, O. (2009). *Portfolios of the Poor: How the World's Poor Live on \$2 a Day*. Princeton University Press. (chapitres sur le crédit informel et les boutiquiers)
- Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann. (heuristiques, SUS)
- Brooke, J. (1996). *SUS: A quick and dirty usability scale*. In *Usability Evaluation in Industry*.
- Povey, D. et al. (2011). *The Kaldi Speech Recognition Toolkit*. IEEE ASRU. (base de Vosk)
- Radford, A. et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision* (Whisper). OpenAI.

## Rapports et données

- GSMA. *State of the Industry Report on Mobile Money* (édition la plus récente).
- BCEAO. *Rapport annuel sur la situation de l'inclusion financière dans l'UEMOA*.
- INStaD Bénin. *Enquête sur le secteur informel / RGPH* (données démographiques et commerce).
- Banque mondiale. *Global Findex Database* (inclusion financière, Bénin).

## Documentation technique

- Flutter — <https://docs.flutter.dev>
- Dart — <https://dart.dev/guides>
- Provider — <https://pub.dev/packages/provider>
- Hive — <https://docs.hivedb.dev>
- speech\_to\_text — [https://pub.dev/packages/speech\\_to\\_text](https://pub.dev/packages/speech_to_text)
- url\_launcher — [https://pub.dev/packages/url\\_launcher](https://pub.dev/packages/url_launcher)
- Android Intents ( ACTION\_SENDTO , ACTION\_DIAL ) — <https://developer.android.com/guide/components/intents-common>
- Material Design 3 — <https://m3.material.io>
- Vosk — <https://alphacephei.com/vosk>

## Solutions comparées

- OkCredit — <https://okcredit.in> · Khatabook — <https://khatabook.com> · Kippa — <https://kippa.africa> · Weebi — <https://weebi.com>

# 13. Annexes

## A. Guide d'installation et d'exécution

```
git clone https://github.com/User26003/ardoise-project.git
cd ardoise-project
flutter pub get
flutter analyze && flutter test          # qualité + tests
flutter run -d chrome                    # démo Web
flutter build apk --release               # APK Android → build/app/outputs/flutter-apk/
```

Pré-requis : Flutter 3.35.x, Dart 3.9.x, Android SDK 35, JDK 17.

## B. Formats de phrases vocales reconnus

| Phrase dite                            | Client | Article | Montant | Type     |
|----------------------------------------|--------|---------|---------|----------|
| « Codjo, riz, 500 »                    | Codjo  | Riz     | 500     | Crédit   |
| « Afi a pris de l'huile à 800 francs » | Afi    | Huile   | 800     | Crédit   |
| « Nadège, savon, mille deux cents »    | Nadège | Savon   | 1 200   | Crédit   |
| « Kossi, gaz, 6 500 »                  | Kossi  | Gaz     | 6 500   | Crédit   |
| « Rachid a payé 2000 »                 | Rachid | —       | 2 000   | Paiement |
| « Sènan a remboursé cinq mille »       | Sènan  | —       | 5 000   | Paiement |

## C. Palette et sémantique des couleurs

| Couleur       | Hex     | Usage                      |
|---------------|---------|----------------------------|
| Orange        | #F26A1B | Primaire, crédit, « doit » |
| Rouge profond | #D9381E | Dégradé d'en-tête          |
| Rouge retard  | #D62839 | Client en retard           |
| Ambre         | #FFB300 | Accent, promesse à venir   |
| Vert          | #1E9E63 | Paiement, soldé            |
| Turquoise     | #00A6A6 | Rappel SMS                 |
| Crème         | #FFF6EC | Fond                       |
| Encre         | #2B1B12 | Texte                      |

## D. Données de démonstration (seed)

Six clients (Codjo, Afi, Sènan, Rachid, Nadège, Kossi) avec crédits et paiements sur 30 jours ; Afi avec une promesse **dépassée** (retard), Sènan promet **demain**, Rachid dans **2 jours**. Le seed n'est injecté qu'au premier lancement si la base est vide ( `demoSeeded` ).

## E. Questionnaire d'enquête terrain (proposition)

1. Depuis combien d'années tenez-vous cette boutique ?
2. Combien de clients vous doivent de l'argent en ce moment ? Montant total estimé ?
3. Comment notez-vous les dettes aujourd'hui ? Montrez-moi.
4. Avez-vous déjà perdu de l'argent à cause d'un oubli / d'un cahier perdu ? Combien ?
5. Comment rappelez-vous à un client qu'il doit payer ? Est-ce gênant ?
6. Vos clients vous donnent-ils une date pour payer ? La respectent-ils ?
7. Quel téléphone avez-vous ? Avez-vous internet ? Combien par semaine ?
8. Utilisez-vous Mobile Money pour être payé ?
9. Savez-vous lire et écrire le français ? Quelle langue parlez-vous avec vos clients ?

10. Si une application notait les dettes quand vous parlez, l'utiliseriez-vous ? Qu'en attendriez-vous ?

## F. Grille SUS (System Usability Scale) — 10 items, échelle 1-5

À administrer après le test d'utilisabilité ; score /100 ; > 68 = au-dessus de la moyenne.

## G. Captures d'écran à insérer

- Onboarding · Accueil (total, filtres, liste avec dates et promesses) · Feuille micro en écoute · Formulaire pré-rempli · Fiche client avec promesse · Sélecteur de date · Statistiques · Réglages · Aperçu SMS · Vue tablette (grille 2 colonnes).

## H. Journal de développement (Git)

| Commit  | Contenu                                                                                                                                           |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 87806a4 | v1.0 — Carnet hors ligne : saisie vocale, clients, paiements, rappels SMS, stats, onboarding, icône, config Android                               |
| f5921eb | v1.1 — Date promise de remboursement (retard dès le lendemain), bannière promesses, SMS adapté, interface responsive, tests de la règle de retard |

---

*Document généré à partir du code source réel du projet Ardoise. À adapter, enrichir par vos données de terrain et relire avec votre encadrant.*